Universidade Cruzeiro do Sul
Ciência da Computação

Aula 8 - Ponteiros

Disciplina: Paradigmas de Linguagens de Programação
Professor: Álvaro Colombero

Objetivos



- Compreender o que são ponteiros e como funcionam.
- Revisar o conceito de passagem de parâmetros por valor e por referência e relacioná-los com ponteiros.
- Diferenciar alocação estática de alocação dinâmica e a identificar a sua relação com ponteiros.

Ponteiros

- Ponteiros
  - Um valor que representa um endereço de memória.
  - São muito utilizados em C e C++ e Perl, mas outras linguagens como Pascal e Ada também suportam a manipulação explícita de ponteiros.
  - Java, C# e outras linguagens também utilizam ponteiros, mas de forma implícita.
  - A utilização correta de ponteiros deixa o programa mais rápido.
  - Ponteiros são a base para a criação de estruturas de dados mais avançadas, como listas, pilhas, filas, árvores.

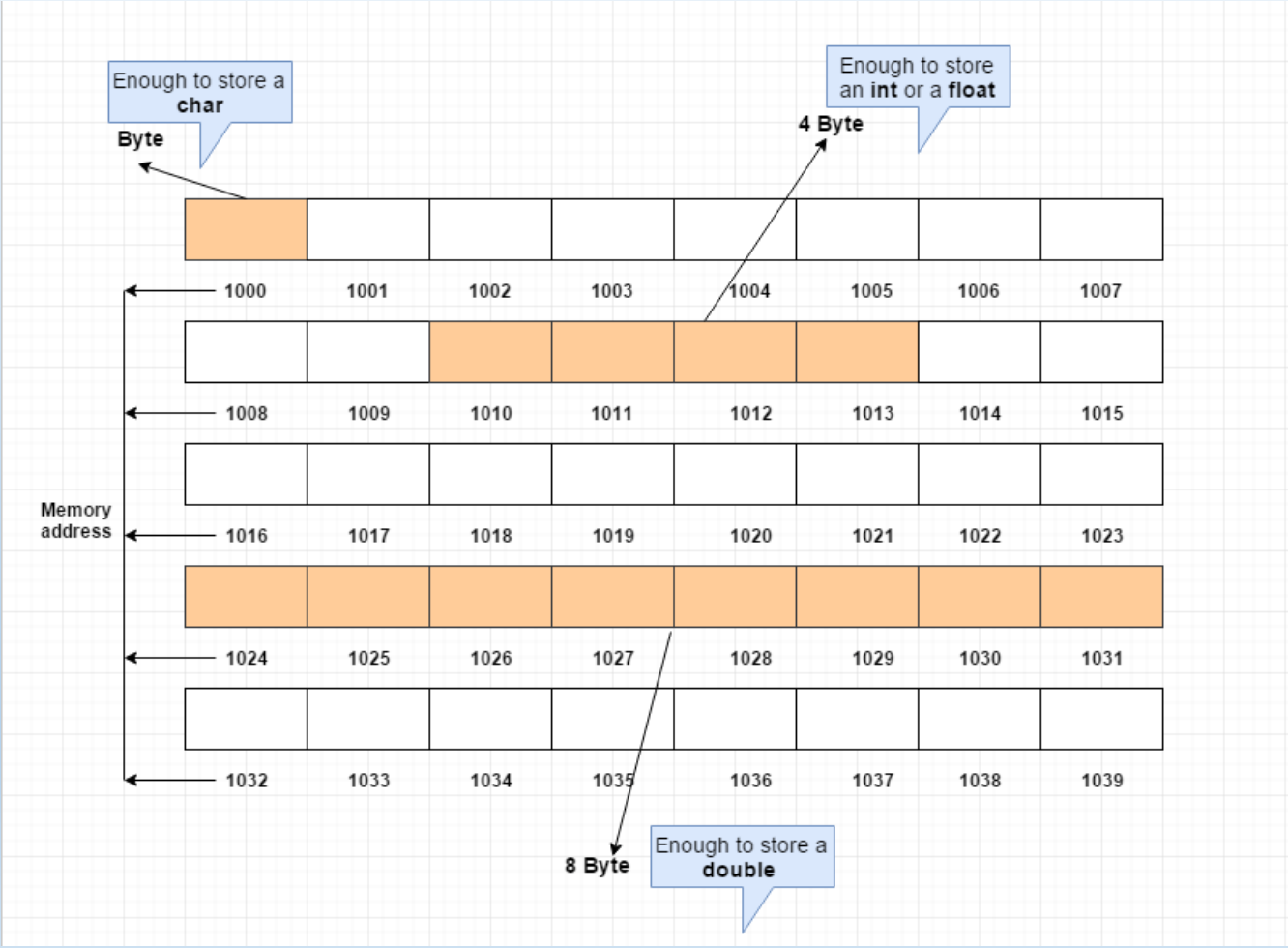
Ponteiros

O que representa os valores apresentados por este programa?

```
#include <stdio.h>
int main(void) {
    int x=2147483647;
    printf("%d \n", x);
    printf("%ld \n", &x);
  return 0;
}
```

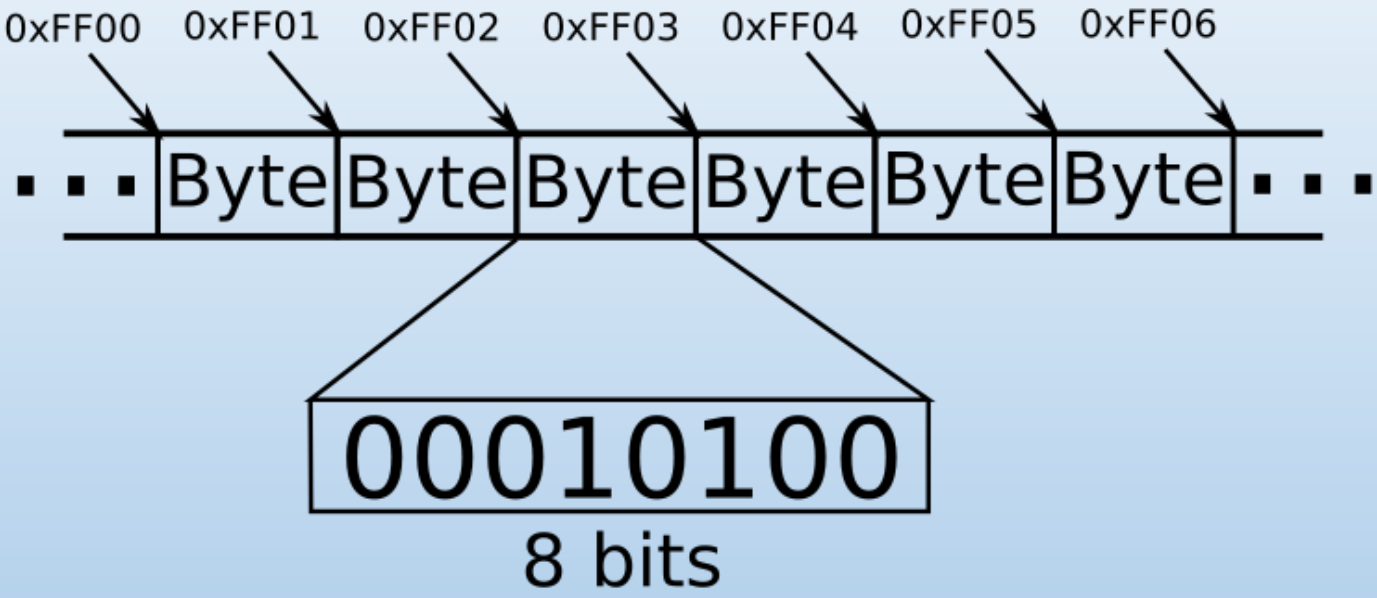
Ponteiros

Organização da memória do computador.



Ponteiros

Organização da memória do computador.



Ponteiros

De que forma os parâmetros podem ser passados para uma função?

Qual a diferença entre elas?



Passagem Parâmetros

- Passagem de parâmetro por valor:
  - O valor do parâmetro real é passado para o parâmetro formal (entrada).
  - Realiza uma cópia do valor, o que demanda mais espaço de armazenamento.
- Passagem de parâmetro por referência:
  - Em vez de copiar os valores do parâmetro atual para o formal, o parâmetro formal oferece um caminho (ponteiro) de acesso ao dado existente no parâmetro atual.

Exemplo 1: Quais valores são apresentados pelo código ao lado?



```
using System;

class Program {

  static void Metodo1(int x) {
    x = x + 1;
    Console.WriteLine(x);
  }

    static void Metodo2(ref int x) {
    x = x + 1;
    Console.WriteLine(x);
  }

  public static void Main (string[] args) {
    int numero = 0;
    Console.WriteLine(numero);
    Metodo1(numero);
    Console.WriteLine(numero);
    Metodo2(ref numero);
    Console.WriteLine(numero);
  }
}
```

Por que isto ocorre?

```java
import java.util.*;

public class Main {
  public static void imprime (int vet[], int tam) {
    for (int i=0;i<tam;i++) {
        System.out.println("Vetor[" + i + "] = " + vet[i]);
        vet[i]++;
    }
    tam++;
}

  public static void main(String[] args) {
    int vetor[] = {1,3,5,7,9};
    int tamanho = 4;
    System.out.println("Impressao 1");
    imprime (vetor,tamanho);
    System.out.println("Impressao 2");
    imprime (vetor,tamanho);

  }
}
```



Exemplo 2: Quais valores são apresentados pelo código ao lado?

Por que isto ocorre?

```java
class Main {
  public static void main(String[] args) {
    Pessoa p1, p2;
    p1 = new Pessoa("Joao");
    p2 = p1;
    System.out.println(p1.nome);
    System.out.println(p2.nome);
    p1.nome = "Jose";
    System.out.println(p1.nome);
    System.out.println(p2.nome);
    p2 = new Pessoa("Maria");
    System.out.println(p1.nome);
    System.out.println(p2.nome);
    p1.nome = "Joao";
    System.out.println(p1.nome);
    System.out.println(p2.nome);
  }
}

class Pessoa {
  public String nome;

  public Pessoa(String nome){
    this.nome = nome;
  }
}
```



Exemplo 3: Quais valores são apresentados pelo código ao lado?

Por que isto ocorre?

Ponteiros

- Como visto anteriormente, o tempo de vida de uma variável é fortemente influenciada pelo seu escopo, que pode ser dividido em:
  - Global.
  - Local.
- Outro fator relacionado ao tempo de vida e ao local onde esta variável será armazenada na memória é a forma como ela será alocada, que pode ser:
  - Alocação estática.
  - Alocação dinâmica.

Ponteiros

- A forma como uma variável é alocada determina onde ela será armazenada na memória:
  - Alocação estática: realizada em tempo de compilação e seu tamanho não pode ser alterado.
  - Alocação dinâmica: realizada em tempo de execução e seu tamanho pode ser alterado.



```cpp
#include <iostream>
using namespace std;

int n, res;
int funcaoA(int x, int y){
  int resultado;
  resultado = x + y;
  cout << "Endereco de x: " << &x <<"\n";
  cout << "Endereco de y: " << &y <<"\n";
  return resultado;
}

int main() {
  int a, b;
  cout << "Digite um número inteiro: \n";
  cin >> a;
  cout << "Digite um número inteiro: \n";
  cin >> b;
  cout << "Endereco de a: " << &a <<"\n";
  cout << "Endereco de b: " << &b <<"\n";
  cout << "Endereco de n: " << &n <<"\n";
  cout << "Endereco de res: " << &res <<"\n";
  res = funcaoA(a,b);
  cout << "Digite a quantidade de elementos do vetor:\n";
  cin >> n;
  int* vetor = new int(4);
  cout << "Endereco de vetor: " << vetor <<"\n";
  for (int i=0;i<n;i++) {
      cout << "Digite o elemento da posicao " << i << ": \n";
      cin >> vetor[i];
  }
  for (int i=0;i<n;i++) {
    cout << "Vetor[" << i <<"] = " << vetor[i]*res << endl;
  }
  delete vetor;
  return 0;
}
```

Exemplo 4: Execute o código abaixo e verifique os endereços de cada uma das variáveis.

Analise quais estão armazenadas no mesmo espaço de memória e quais estão separadas.